

HW11 (Ch11) Solution 補充：

GitHub Commit 步驟

對應 Problem 8 動手實驗 (vLLM Prefix Cache + RAG with Chroma)

概要：為什麼要 commit 到 GitHub？

Problem 8 要求把實驗結果 commit 到自己的 GitHub repo 並附 commit URL。這是業界與學術界用來「證明此程式真的是你寫的、且時間戳記不可竄改」的標準做法。三件事學會後一輩子都會用：

- 建立 Git repo 與遠端 GitHub repo
- 把檔案 commit、push 到 GitHub
- 取得 commit URL (給作業繳交用)

💡 整套流程約需 30 分鐘 (含註冊 + 安裝)，每位同學第一次做。第二次以後只需 5 分鐘。

第一部分：一次性環境設定 (每位同學第一次做)

Step 1. 註冊 GitHub 帳號

1. 前往 <https://github.com/signup>
2. 用學校 email 註冊 (例如 你的學號@ulive.pccu.edu.tw 或個人 email 都可以)
3. 設定 username (建議用「英文名 + 學號末 4 碼」，例如 alice1234，方便老師辨識)
4. 完成 email 驗證
5. 登入後保留瀏覽器分頁

學生方案 (GitHub Education) 可免費獲得 GitHub Pro 與 Copilot：
<https://education.github.com/pack> 用學校 email 申請即可。

Step 2. 安裝 Git

Windows 11 用戶請依下列步驟：

1. 開瀏覽器到 <https://git-scm.com/download/win>
2. 下載 64-bit Git for Windows (自動偵測 OS 後直接下載)
3. 執行 .exe 檔案，全部按「Next」採用預設值即可 (首次安裝者勿亂改設定)
4. 安裝完成後，按 Win 鍵搜尋 Git Bash，打開後出現黑色終端機代表成功，或者也可以用 PowerShell 確認 Git 是否裝好：

```
git --version
```

預期看到 (版本號可能不同)：

```
git version 2.45.1.windows.1
```

若出現「找不到 git 指令」，重開機後再試一次 (PATH 環境變數需重整)。

Step 3. Git 初次設定 (告訴 Git 你是誰)

在 Anaconda Prompt 或 Git Bash 執行 (把雙引號內換成自己的)：

```
git config --global user.name "Your Name (學號)"
git config --global user.email "your_email@example.com"
```

email 必須與 GitHub 帳號相同 (否則 push 後 commit 不會顯示在你的個人頁)。

確認設定：

```
git config --global user.name
git config --global user.email
```

Step 4. 設定 GitHub 認證 (推薦：Personal Access Token)

GitHub 從 2021 年起不接受密碼登入，必須用 Personal Access Token (PAT)：

1. 登入 GitHub，點右上角頭像 → **Settings**
2. 左側選單最下方：**Developer settings**
3. 左側：**Personal access tokens** → **Tokens (classic)**
4. 點 “**Generate new token (classic)**”
5. Note 欄填：「Algorithms HW11 token」 (或其他你能辨識的名稱)
6. Expiration：**90 days** (或更長)
7. 勾選範圍 (scopes)：「**repo**」 (包含 public_repo, repo:status 等所有 repo 權限)

8. 點底部 “Generate token”

9. 複製顯示的 token（只會出現一次！）儲存到記事本暫存

這個 token 等同密碼，不可貼給任何人或上傳到 GitHub。後面 git push 時會被問帳號密碼，貼這個 token 當「密碼」。

第二部分：上傳 vLLM 實驗到 GitHub

Step 5. 在 GitHub 建立新 repo

1. 登入 GitHub，點左上角綠色「+」→ **New repository**
2. Repository name 填：algo-hw11-vllm-prefix-cache
3. Description（選填）：Algorithms HW11 - vLLM Prefix Caching Experiment
4. 選 **Public**（公開，老師才能看；若需私人，後面要把老師加為 collaborator）
5. 勾選「Add a README file」
6. .gitignore 選「Python」（避免上傳虛擬環境）
7. License 可選「MIT License」
8. 點綠色「Create repository」
9. 你會看到 repo 的網址，類似 <https://github.com/yourname/algo-hw11-vllm-prefix-cache>

Step 6. 把實驗檔案 clone 到本機

在 Anaconda Prompt 或 Git Bash：

```
cd [Your path to this assignment]
git clone https://github.com/[yourname]/algo-hw11-vllm-prefix-cache.git
cd algo-hw11-vllm-prefix-cache
```

（把 yourname 換成你的 GitHub username）

預期看到：

已複製到 .../algo-hw11-vllm-prefix-cache/

Step 7. 把 README-11-p76.md 提到的 5 個檔案放進來

依 README-11-p76.md 第 6 節「檔案清單」，把以下 5 個檔案複製到 repo 資料夾：

- experiment.py (主腳本)
- plot_results.py (畫圖腳本)
- run_compare.sh (Linux/Colab 一鍵跑)
- run_compare.ps1 (Windows 一鍵跑)
- requirements.txt

再加上你跑出的結果檔：

- comparison.png (plot_results.py 產出的圖)
- result_off.json、result_on.json (experiment.py 跑兩次的原始資料)
- output.txt (終端機輸出，含「===== 比較摘要 =====」段落)

Step 8. 修改 README.md (必填項目)

編輯 repo 根目錄的 README.md，至少包含：

Algorithms HW11 – vLLM Prefix Caching

學生資訊

- 姓名：[你的姓名]
- 學號：[你的學號]
- 課程：3468 演算法 1142

實驗環境

- 平台：Google Colab T4 / WSL2 / 純 Linux (擇一)
- vLLM 版本：[填 vllm.__version__]
- 模型：Qwen2.5-0.5B-Instruct (或你實際用的)
- prompts 數：100
- max_tokens：64

結果摘要

(從 output.txt 貼上比較摘要的 5 行)

結論

(200 字描述觀察到的現象與 §11.4 連結)

對應作業

- 作業：3468 演算法 HW11 (Ch11) Problem 8(a)

- 投影片：CLRS 4e Ch11 v4 PPT 第 82 頁
- 實驗指南：本 repo 內 README-11-p76.md

Step 9. 第一次 commit + push

依序執行：

```
git add .
```

預期：沒有輸出（成功時靜默）。如果出錯，先檢查目前路徑：

```
pwd
```

應該顯示 `.../algo-hw11-vllm-prefix-cache`。

```
git status
```

預期看到一堆綠色檔名（待 commit 的新檔）：

```
On branch main
```

```
Changes to be committed:
```

```
new file:   experiment.py
new file:   plot_results.py
new file:   comparison.png
...
```

```
git commit -m "Add vLLM prefix cache experiment results for HW11 Problem 8(a)"
```

預期：

```
[main 1a2b3c4] Add vLLM prefix cache experiment results for HW11 Problem 8(a)
8 files changed, 234 insertions(+)
create mode 100644 experiment.py
...
```

```
git push origin main
```

第一次 push 會跳出登入視窗（Git Credential Manager），輸入：

- **Username**：[你的 GitHub username]
- **Password**：[貼上 Step 4 產生的 PAT，不是 GitHub 密碼]

預期看到：

```
Enumerating objects: 12, done.
```

```
...
```

```
To https://github.com/yourname/algo-hw11-vllm-prefix-cache.git
abcd123..1a2b3c4  main -> main
```

Step 10. 取得 commit URL (作業繳交用)

1. 開瀏覽器到你的 repo 頁面：<https://github.com/yourname/algo-hw11-vllm-prefix-cache>
2. 點上方「Commits」按鈕（在 Code 標籤旁邊），會看到 commit 歷史
3. 點剛剛 push 的 commit「Add vLLM prefix cache experiment...」
4. 瀏覽器網址列現在的 URL 就是 commit URL，類似：

[https://github.com/\[yourname\]/algo-hw11-vllm-prefix-cache/commit/1a2b3c4d5e6f...](https://github.com/[yourname]/algo-hw11-vllm-prefix-cache/commit/1a2b3c4d5e6f...)

複製整段 URL，這就是作業要附的 commit URL。

第三部分：上傳 RAG/Chroma 實驗到 GitHub

與 Part 2 流程幾乎一樣，差別只在 repo 名稱與內容檔案。

Step 11. 建立第二個 repo

- Repository name: algo-hw11-rag-chroma
- Description: Algorithms HW11 - RAG with LangChain + Chroma
- 其他設定同 Step 5

Step 12. clone + 加檔案

```
cd [Your path to this assignment]
git clone https://github.com/yourname/algo-hw11-rag-chroma.git
cd algo-hw11-rag-chroma
```

依 README-ch11-p79.md 加入：

- data/clrs_ch11.md (範例語料)
- ingest.py、query.py、eval.py
- requirements.txt
- eval_output.txt (python eval.py 的輸出，含 recall@3 = X/11 = X%)
- sample_queries.txt (你的 4 個中英文 query 與 top-3 距離結果)

不要把 chroma_db/ 資料夾傳上去！它是本機 Chroma 資料庫，太大且每台機器重新生成。
在 .gitignore 加入 chroma_db/ 一行。

Step 13. 同樣的 commit + push 流程

```
git add .  
git commit -m "Add RAG with Chroma experiment for HW11 Problem 8(b)"  
git push origin main
```

取得 commit URL (同 Step 10)。

第四部分：作業繳交格式

在你的 HW11 答卷檔案 (PDF 或 docx) 的 Problem 8 部分，直接附上兩個 commit URL：

Problem 8 GitHub Commit URLs

(a) vLLM Prefix Cache 實驗：

<https://github.com/yourname/algo-hw11-vllm-prefix-cache/commit/1a2b3c4d5e6f...>

(b) RAG with Chroma 實驗：

<https://github.com/yourname/algo-hw11-rag-chroma/commit/abcdef0123456...>

老師收到作業後會點開兩個 URL 檢查：

- 提交時間是否在繳交期限前
- commit author 是否與作業署名一致
- README.md 是否含學生資訊
- 結果檔 (comparison.png、eval_output.txt) 是否真的存在

結語

Git + GitHub 的學習曲線約 1-2 小時，但這是資工系畢業生必備的基本功。第一次做完之後，你的 GitHub 個人頁就有了第一個 commit，是日後求職、研究所申請、開源貢獻的起點。

完成本作業後，建議把以下兩個 repo 都設為 Public，作為你的 portfolio：

- algo-hw11-vllm-prefix-cache：證明你做過 vLLM 推論優化實驗
- algo-hw11-rag-chroma：證明你做過 RAG 系統實作

面試時可以指著這兩個 repo 說：「我大二修演算法時實作過這兩個 production-grade 的 LLM 系統」—這比口頭描述強得多。